LAB REPORT: DESIGN AND IMPLEMENTATION OF A 32-BIT MIPS CPU USING VHDL

Course: CPEN 315 – Computer Organization and Architecture

Members

GODFRED SAFO OFORI – 22046644

CHRISTOPHER Y.E MAWULI - 22071976

ABSTRACT

This project presents the design and implementation of a **32-bit single-cycle MIPS CPU** using **VHDL (VHSIC Hardware Description Language)**. The system was developed by extending a 16-bit processor architecture into a more robust 32-bit design. Core components such as the **ALU, register file, control unit, instruction memory, and data memory** were implemented and tested using simulation tools. Functional verification was achieved through testbenches, validating arithmetic, logical, memory, and control operations. The results demonstrate correct execution of MIPS instructions and highlight the effectiveness of hardware description languages in processor design.

INTRODUCTION

Modern processors rely on efficient hardware design to execute instructions. This project focuses on designing a **32-bit MIPS processor**, a simplified RISC architecture widely used in academic settings.

Objectives

- Design a 32-bit CPU using VHDL
- Understand datapath and control unit interaction
- Implement and simulate processor components
- Validate functionality using testbenches

Relevance

This project bridges theory and practice by translating **computer architecture concepts** into a working hardware model.

ARCHITECTURE DESIGN

The CPU follows a **single-cycle MIPS architecture**, meaning each instruction executes in one clock cycle.

Major Components

- **Arithmetic Logic Unit (ALU)**
- **Register File (8 registers)**
- **Control Unit**
- **Instruction Memory**
- **Data Memory**
- **Program Counter (PC)**

Datapath Overview

The datapath connects all components and enables:

- Instruction fetch
- Decode
- Execute
- Memory access
- Write-back

INSTRUCTION SET ARCHITECTURE (ISA)

The processor supports a subset of MIPS instructions:

R-Type Instructions

- ADD, SUB, AND, OR, SLT

I-Type Instructions

- LW, SW, BEQ, ADDI, SLTI

J-Type Instructions

- J, JAL, JR

Each instruction consists of fields such as:

- Opcode
- Source registers (rs, rt)

- Destination register (rd)
- Immediate/Address

IMPLEMENTATION DETAILS

Data Memory

- 32-bit word size
- Synchronous read/write
- Controlled by mem_read and mem_write

Test Case Results:

- Write 1024 → Address 2 → Read successful
- Write 429496 → Address 4 → Read successful

Arithmetic Logic Unit (ALU)

Performs arithmetic and logical operations based on control signals.

Operations Tested:

- 2500 + 25000 = 27500
- 540250 – 37800 = 502450
- AND(53957, 30000)
- OR(746353, 846465)
- SLT(58847537, 72464383) → Output = 1

4.3 Register File

- 8 registers (32-bit each)
- Dual read, single write

Test Data:

- R1 = 1,934,858
- R3 = 8,558,447

- R5 = 203,848,544
- R7 = 20,670,420

All values were correctly written and read.

Instruction Memory

- Stores program instructions
- Indexed by Program Counter

Sample Instructions:

- add $t0, $t1, $t2
- sub $t2, $t2, $t3
- and $t1, $t2, $t0
- or $t2, $t3, $t1

All instructions were correctly fetched and decoded.

Control Unit

Generates control signals based on opcode.

Control Signals Include:

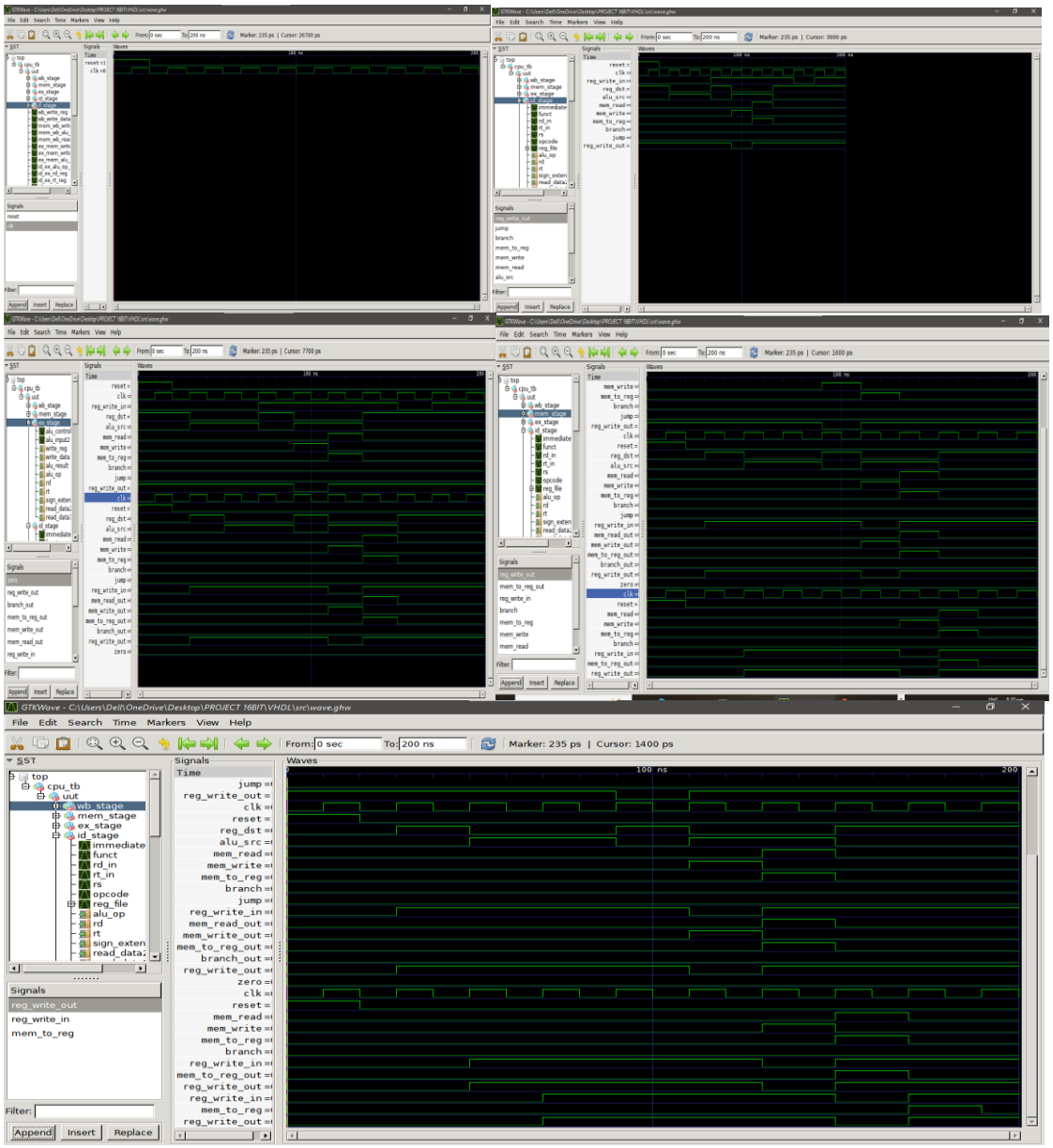
- RegDst
- MemRead
- MemWrite
- ALUOp
- Branch
- Jump

Verified Instructions:

- add
- slti

- j
- jal
- lw
- sw
- beq
- addi

SIMULATION AND RESULTS



Observed Results

- Correct execution of arithmetic operations
- Accurate memory read/write
- Proper instruction decoding
- Correct branching and jumping behavior

Performance

- Single-cycle execution ensures simplicity
- Trade-off: longer clock period due to full instruction execution in one cycle

CHALLENGES AND SOLUTIONS

Challenge	Solution
Bit-width mismatch	Standardized all signals to 32-bit
Control signal errors	Debugged using waveform analysis
Memory addressing issues	Corrected indexing logic
Instruction encoding errors	Verified using MIPS format

CONCLUSION

The project successfully implemented a **32-bit MIPS CPU using VHDL**. All components functioned correctly and were validated through simulation. The design demonstrates a solid understanding of processor architecture and digital system design.

REFERENCES

[1] Patterson, D. A., & Hennessy, J. L. (2014). Computer Organization and Design: The Hardware/Software Interface (5th ed.). Morgan Kaufmann Publishers.

[2] IEEE Standard VHDL Language Reference Manual. IEEE Std 1076-2008. Institute of Electrical and Electronics Engineers.

[3] Zwolinski, M. (2004). Digital System Design with VHDL (2nd ed.). Pearson Education Limited.

[4] Weste, N., & Harris, D. (2010). CMOS VLSI Design: A Circuits and Systems Perspective (4th ed.). Addison-Wesley.

[5] CPEN 315 Course Lecture Notes and Final Project Specification, First Semester 2025/2026. Department of Computer Engineering, University of Ghana.